

Gradient-Based Hyperparameter Optimization Over Long Horizons

Paul Micaelli, Amos Storkey

Talk by: Fedor Sobolevsky

NeurIPS 2021

The Problem: Hyperparameter Optimization (HPO)

- ▶ Deep learning success depends on **carefully tuned hyperparameters**: learning rate schedules, momentum, weight decay, etc.
- ▶ Current approaches have significant limitations:
 - ▶ **Black-box methods** (Random Search, Bayesian Optimization, HyperBand): Slow trial-and-error
 - ▶ **Greedy gradient-based methods**: Fast but suboptimal due to "short-horizon bias"
- ▶ Key challenge: Long training horizons ($\sim 10^4$ gradient steps)
 - ▶ Memory scales with horizon length
 - ▶ Hypergradients degrade/vanish through long unrolled optimization

Problem Formulation

Bilevel Optimization

Find hyperparameters λ^* that minimize validation loss after T training steps:

$$\begin{aligned} \lambda^* &= \arg \min_{\lambda} \mathcal{L}_{\text{val}}(\boldsymbol{\theta}_T(\lambda), \mathcal{D}_{\text{val}}) \\ \text{s.t. } \theta_{t+1} &= \Phi(\mathcal{L}_{\text{train}}(\boldsymbol{\theta}_t, \mathcal{D}_{\text{train}}), \lambda_{[t+1]}) \end{aligned}$$

- ▶ $\boldsymbol{\theta}_t$: network weights at step t
- ▶ $\lambda_{[t]}$: hyperparameter at step t (e.g., learning rate)
- ▶ Φ : optimizer (SGD with momentum)
- ▶ Goal: Compute hypergradient $d\mathcal{L}_{\text{val}}/d\lambda$

Greedy vs. Non-greedy HPO

Greedy ($H \ll T$)

- ▶ Split into small horizons
- ▶ Update λ online
- ▶ Memory: $\mathcal{O}(FH)$

Pro: Fast, stable gradients

Con: Solves wrong objective

Non-greedy ($H = T$)

- ▶ Single horizon = full training
- ▶ Update λ after T steps
- ▶ Memory: $\mathcal{O}(FT)$ with BPTT

Pro: True objective

Con: Memory explosion, gradient degradation

Goal: Combine strengths of both paradigms

Forward-Mode Differentiation

Alternative to BPTT with better memory scaling

Memory: $\mathcal{O}(DN)$ where:

- ▶ D : number of weights
- ▶ N : number of learnable hyperparameters

Recursive update for hypergradients:

$$Z_t = A_t Z_{t-1} + B_t \quad \text{where} \quad Z_t = \frac{d\theta_t}{d\lambda}$$

For SGD with momentum (learning rate α):

$$A_t^\alpha = 1 - \alpha_t \left(\frac{\partial^2 \mathcal{L}_{\text{train}}}{\partial \theta_{t-1}^2} + \xi_t \mathbf{1} \right)$$
$$B_t^\alpha = -\beta_t \alpha_t C_{t-1}^\alpha - \delta_t^\otimes \left(\beta_t \nu_{t-1} + \frac{\partial \mathcal{L}_{\text{train}}}{\partial \theta_{t-1}} + \xi_t \theta_{t-1} \right)$$

Key Challenge: Gradient Degradation

- ▶ Hypergradients are **noisy** due to:
 - ▶ Minibatch ordering
 - ▶ Weight initialization θ_0
- ▶ Noise compounds over long horizons
- ▶ Ideal: Ensemble average over many seeds
- ▶ Reality: Too expensive ($\mathcal{O}(T \times \text{seeds})$)

Idea: Use time averaging instead of ensemble averaging

FDS: Forward-mode Differentiation with Sharing

Key Insight

Share hyperparameters over contiguous time windows W

Two effects:

1. **Noise reduction:** Average over W noisy hypergradients
2. **Bias increase:** Approximate W different means with one value

Theorem 4.1

For hypergradients $\mathbf{g} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with Lipschitz $\mu(\varepsilon)$ and max correlation c :

$$\text{MSE}_W \leq \frac{1 + c(W - 1)}{W} \text{MSE}_1 + \varepsilon^2 \frac{W^2 - 1}{12}$$

FDS Algorithm

Initialize: $N^\alpha, W = T/N^\alpha, \alpha = \mathbf{0}^{N^\alpha}$

#outer loop

for o in $1, 2, \dots$ **do**

Initialize: $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \theta_0 \in \mathbb{R}^D$,
 $\mathbf{Z}^\alpha = \mathbf{0}^{D \times N^\alpha}, \mathbf{C}^\alpha = \mathbf{0}^{D \times N^\alpha}$

#inner loop

for t in $1, 2, \dots, T$ **do**

$\mathbf{x}_{\text{train}}, \mathbf{y}_{\text{train}} \sim \mathcal{D}_{\text{train}}$

$\mathbf{g}_{\text{train}} = \partial \mathcal{L}_{\text{train}}(\mathbf{x}_{\text{train}}, \mathbf{y}_{\text{train}}) / \partial \theta$

#hyperparameter sharing

$i = \lceil t/W \rceil$

$\mathcal{H}\mathbf{Z}_{[1:i]}^\alpha = \partial(\mathbf{g}_{\text{train}} \mathbf{Z}_{[1:i]}^\alpha) / \partial \theta$

$\mathbf{Z}_{[1:i]}^\alpha = \mathbf{A}^\alpha \mathbf{Z}_{[1:i]}^\alpha + \mathbf{B}_{[1:i]}^\alpha$

update $\mathbf{C}^\alpha$ (Eq 4)

$\theta_{t+1} = \Phi(\theta_t, \mathbf{g}_{\text{train}})$

end for

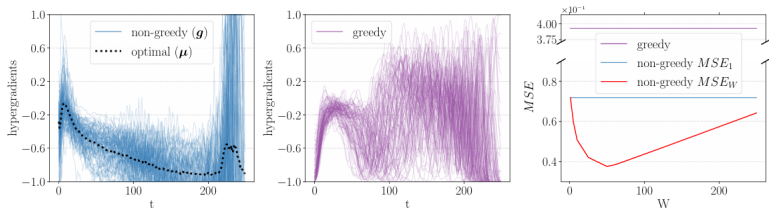
$\mathbf{g}_{\text{val}} = \partial \mathcal{L}_{\text{val}}(\mathcal{D}_{\text{val}}) / \partial \theta$

$\alpha \leftarrow \alpha - 0.1 \times \mathbf{g}_{\text{val}} \mathbf{Z}^\alpha / W$

end for

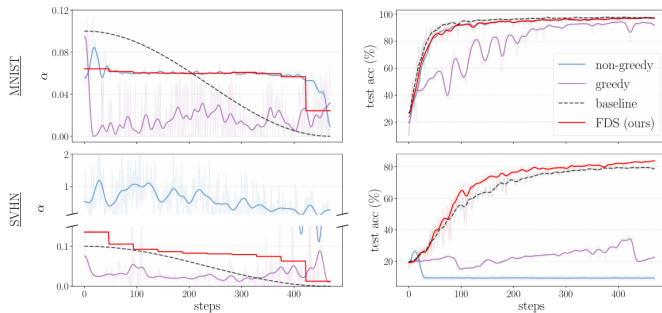
- ▶ Learn $N^\alpha = T/W$ learning rates
- ▶ Forward-mode with shared columns in $\mathbf{Z}^\alpha$
- ▶ Update hyperparameters once per outer loop (non-greedy!)

Experimental Validation: Hypergradient Noise



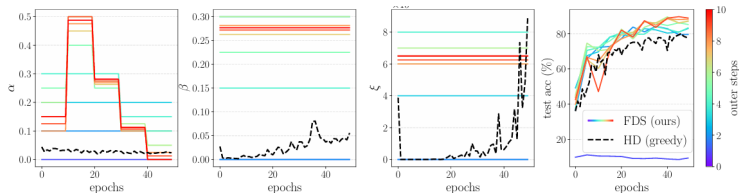
- ▶ LeNet on SVHN, $T = 250$ steps
- ▶ Non-greedy sharing ($W = 50$) reduces MSE significantly
- ▶ Greedy is poor approximation to optimal hypergradients

Learning Rate Schedules



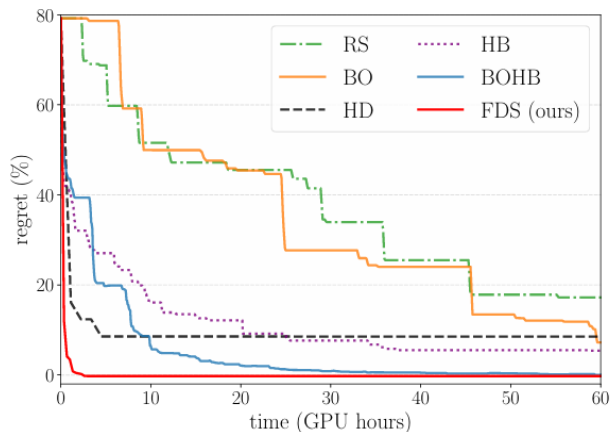
- ▶ MNIST (left): Non-greedy works without sharing
- ▶ SVHN (right): Only FDS (sharing) learns reasonable schedule
- ▶ Outperforms cosine annealing on SVHN

CIFAR-10 Results (WRN-16-1, 50 epochs)



- ▶ Learning 7 rates ($W = 10$), 1 momentum, 1 weight decay
- ▶ Converges in 10 outer steps
- ▶ Discovers one-cycle-like schedule automatically
- ▶ Significantly outperforms greedy HD [Baydin et al. 2018]

Comparison with State-of-the-Art



- ▶ FDS reaches zero regret in **3 hours**
- ▶ **20× speedup** over best black-box method

Limitations

- ▶ Scales linearly with number of hyperparameters ($\sim 10^3$ max on 12GB GPU)
- ▶ Requires differentiable hyperparameters (vs. black-box)
- ▶ Manual derivation of forward-mode updates

Conclusion

1. Introduced **FDS**: First non-greedy gradient-based HPO for long horizons
2. **Hyperparameter sharing** reduces noise, enables forward-mode
3. **Theoretical guarantee**: MSE bound with noise-bias trade-off
4. **Empirical results**:
 - ▶ Outperforms greedy methods in quality
 - ▶ 20× faster than black-box methods
 - ▶ Works on real problems (CIFAR-10, WideResNet)